

8-Puzzle Problem

Aim

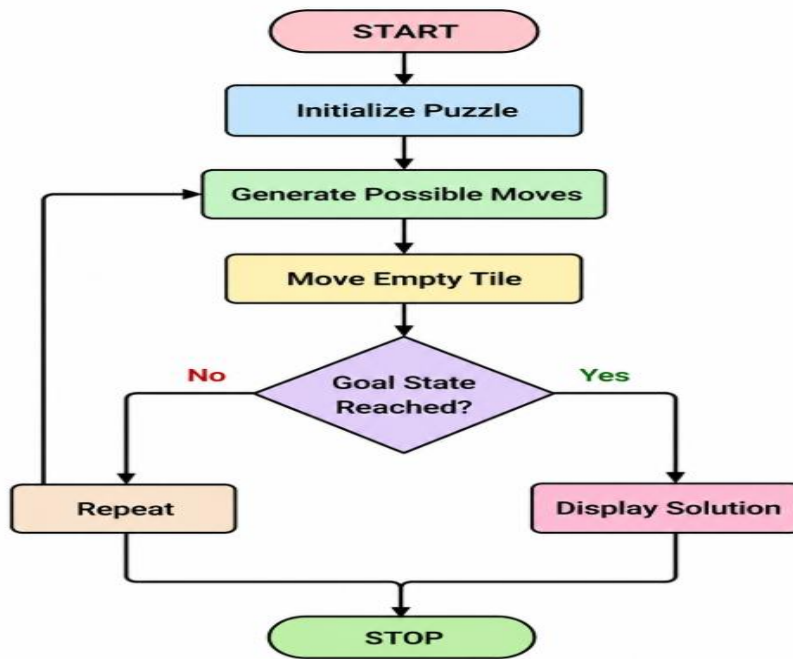
To solve the 8-Puzzle problem using Artificial Intelligence techniques and reach the goal state from the given initial state.

Objectives

- To understand state-space search.
- To apply AI search algorithms.
- To find a sequence of moves to reach the goal state.
- To minimize the number of moves.

Algorithm

1. Start.
2. Define the initial state and goal state.
3. Find the position of the empty tile (0).
4. Generate all possible moves (Up, Down, Left, Right).
5. Move the empty tile to a valid position.
6. Check if the current state matches the goal state.
7. If yes, display the solution.
8. Otherwise, repeat Steps 4–7.



Source code

```
from collections import deque

def solve_puzzle(start, goal):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        if tuple(state) in visited:
            continue

        visited.add(tuple(state))

        zero = state.index(0)

        moves = {
            0: [1, 3],
            1: [0, 2, 4],
            2: [1, 5],
            3: [0, 4, 6],
            4: [1, 3, 5, 7],
            5: [2, 4, 8],
            6: [3, 7],
            7: [4, 6, 8],
            8: [5, 7]
        }
```

OUTPUT:

```
= RESTART: C:/Users/ATHIN/AppData/Local
ssssssssss.py
Steps to reach goal:

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Result

The 8-Puzzle problem was successfully solved using the Breadth First Search (BFS) algorithm. The algorithm found the sequence of moves required to transform the initial state into the goal state.